Task B Guide: How to Approach the Class Diagram

▒▒ Purpose

A slice of three classes and one enum from JPacman, used to show how to read the code and decide what to draw. It is not a piece of the solution. Apply the same questions to every class in scope.

TA: Rinkesh Joshi · rinkeshjoshi@cmail.carleton.ca

Notation: Lecture 4 (Structural Models) · Syntax: lab slide "Task B › PlantUML Class Syntax"

▒▒ What you will see

1. The method (5 questions per class) + a quick IDE check for the type
2. The slice: Unit, Square, Pellet, Direction
3. What we read from the code, and why; methods → operations
4. The diagram (PlantUML + render) + cross-check with IntelliJ (Task D)
5. What the example does not decide for you

1: The Method (Every Class in Scope)

1. Open the class. `abstract`? `interface`? `enum`? `extends` / `implements` something?
2. Go through the fields one by one. What is the type?
   - primitive (`int`, `boolean`), `String`, or an enum → attribute in the box
   - another class of the project → association line (never an attribute: Lecture 4, "no other classes are allowed as types")
3. Multiplicity comes from the code, not from the game's story:
   - single reference, set in the constructor, never null → `1`
   - single reference that can be `null` (set later, or cleared) → `0..1`
   - `List<X>`, `Set<X>`, `Map<K, X>`, `X[]` → `0..*` (or `1..*` / a number if the code guarantees it)
   - role name = field name
4. Check the other end. Target has a field pointing back? → bidirectional (plain line, multiplicity at both ends). No? → arrow from the class that holds the field.
5. Operations: public / protected / package ones, with `+ - # ~`, parameters, return type. Mark abstract ones.

> Fields decide the lines, methods fill the box. Only a field (private Square square;) gives an association and a multiplicity. A method's parameter or return type (occupy(target : Square)) does not: it is written in the operations compartment, and at most it is a dependency. This example is about the fields.

Aggregation / composition, dependencies, interfaces come after all fields are placed.

1: Quick Check in Your IDE: Primitive, Data Type, Enum or Class?

Jump to the type's declaration from the field (put the cursor on the type name, e.g. `Direction`):

- IntelliJ: `Ctrl`+click (`Cmd`+click on Mac), or `Ctrl+B` / `Cmd+B`
- VS Code: `Ctrl`+click (`Cmd`+click on Mac), or `F12` (Go to Definition)
- Eclipse: `Ctrl`+click, or `F3`

Ctrl+click does nothing? Mac → `Cmd`+click · VS Code → install the Extension Pack for Java, open the repo folder, wait for "Java: Ready" · IntelliJ → File › Open the folder with `build.gradle`, accept the Gradle import, wait for indexing · Still nothing → the type name is the file name: open `Direction.java` and read its first line, or search the repo (`Ctrl+Shift+F`; Eclipse `Ctrl+H`) for `enum Direction` / `class Direction` / `interface Direction`; no hit in `src/` → library type or primitive.

Then read the first line of what opens:

What you land on	It is	Draw it as
nothing: `int`, `boolean`, `double` are keywords	primitive	attribute
`java.lang` / `java.util`: `String`, `List`, `Map`	library type	`String` → attribute; collection: below
`public enum Direction` in `nl.tudelft.jpacman…`	enumeration	attribute `- direction : Direction`
`class` / `interface` in `nl.tudelft…`	project class	association line

What an enum is: a type with a fixed list of named values and nothing else to construct. `Direction` has exactly four: `NORTH`, `SOUTH`, `WEST`, `EAST`. A field of that type holds one of them.

Collections: `List<Unit>` is a wrapper. Ignore the wrapper, look at what is inside: `Unit` is a project class, so draw an association `Square` — `Unit`. A list holds none or many → `0..*`. (`List<String>` holds a library type → stays an attribute.)

2: The Slice (1/2): Direction and Unit

Only the lines that matter (line numbers from the repo).

```java
// board/Direction.java
public enum Direction { NORTH(0, -1), SOUTH(0, 1), WEST(-1, 0), EAST(1, 0); ... }

// board/Unit.java
public abstract class Unit {
    private Square square;                                  // :15
    private Direction direction;                            // :20
    protected Unit() { this.direction = Direction.EAST; }   // :25  square stays null
    public void setDirection(Direction newDirection) {...}  // :33
    public Direction getDirection() {...}                   // :41
    public Square getSquare() {...}                         // :51
    public boolean hasSquare() { return square != null; }   // :62
    public void occupy(Square target) {... square = target; target.put(this); }   // :73
    public void leaveSquare() {... square.remove(this); square = null; }          // :87
    protected boolean invariant() {...}                     // :103
    public abstract Sprite getSprite();                     // :112
}
```

2: The Slice (2/2): Square and Pellet

```java
// board/Square.java
public abstract class Square {
    private final List<Unit> occupants;                          // :22
    protected Square() { this.occupants = new ArrayList<>(); ... }  // :32  starts empty (:33)
    public Square getSquareAt(Direction direction) {...}         // :45
    public List<Unit> getOccupants() {...}                       // :70
    void put(Unit occupant) {...}                                // :80  package-private
    void remove(Unit occupant) {...}                             // :93  package-private
    public abstract boolean isAccessibleTo(Unit unit);           // :121
}

// level/Pellet.java
public class Pellet extends Unit {
    private final Sprite image;                                  // :16
    private final int value;                                     // :21
    public int getValue() {...}                                  // :37
    @Override public Sprite getSprite() {...}                    // :42
}
```

3: What We Read From It

Field	Decision	Why (the line)
Unit.direction : Direction	attribute - direction : Direction	enum → allowed type
Unit.square : Square	association, role square, 0..1	null :25, set :79, cleared :90
Square.occupants : List<Unit>	association, role occupants, 0..*	empty list :33, put / remove
the two above	one bidirectional association	both hold a field
Pellet.value : int	attribute - value : Integer	primitive
Pellet.image : Sprite	not drawn	Sprite out of scope
Pellet extends Unit	generalization	the extends keyword

> hasSquare() exists because square can be null. That is the evidence for 0..1.
> Methods are not in this table: occupy(target : Square) takes a Square but adds no line and no multiplicity.

3: direction vs square: Same Kind of Thing, Drawn Differently

Both are fields of Unit (a variable every unit object carries). Only the type differs:

```
private Square square;        // :15  type = a class of the project
private Direction direction;  // :20  type = an enum
```

- direction → Direction is an enum → allowed as an attribute type → written inside the box: - direction : Direction
- square → Square is a class → not allowed as an attribute type → drawn as a line to Square. The field's name becomes the role (-square), how many it can hold becomes the multiplicity

▒▒▒ Where 0..1 comes from (the three lines)

```
protected Unit() { this.direction = Direction.EAST; }          // :25  square not assigned → null → 0
public void occupy(Square target) { ... square = target; ... } // :73, statement :79  one square → 1
public void leaveSquare() { ... square = null; ... }           // :87, statement :90  back to none → 0
```

A single reference (never a list) that starts null, gets one, and can go back to null → 0..1. Same reading for Square.occupants: a List<Unit> that starts empty (:33) and grows / shrinks via put / remove → 0..*.

3: Methods → Operations (One Line Each, Inside the Box)

A method goes in the operations compartment: `visibility name(param : Type, …) : ReturnType`

Java	UML operation	Note
`public void occupy(Square target)`	`+ occupy(target : Square) : void`	`public` → `+`
`public boolean hasSquare()`	`+ hasSquare() : Boolean`	`boolean` → `Boolean`
`void put(Unit occupant)`	`~ put(occupant : Unit) : void`	no modifier → `~`
`public abstract Sprite getSprite()`	`{abstract} + getSprite() : Sprite`	abstract → `{abstract}`
`protected boolean invariant()`	`# invariant() : Boolean`	`protected` → `#`

- Parameter names are kept (`target : Square`), types written the UML way (`int` → `Integer`, `boolean` → `Boolean`)
- `void` is written as the return type: `: void`
- `static` → `{static}` in front, like `{abstract}`
- A parameter type that is a project class (`Square`, `Unit`) changes nothing on the lines: see "Fields decide the lines"

3: Things to Notice

Multiplicity

- `0..1` and `0..*` come from what the code allows (null checks, empty constructor), not from "every unit is always somewhere in the game"
- Omitted multiplicity = `1` (Lecture 4). When it is not 1, write it.
- Put the number next to the class it counts

Types

- code `int` → UML `Integer` (UML primitives: Boolean · Integer · Real · String)

Visibility and abstract

- `put` / `remove` have no modifier → package-private → `~`
- `+` public · `-` private · `#` protected
- `abstract` classes and operations are marked
- `getSprite()` is abstract in `Unit`, concrete in `Pellet`

Out of scope

- `Sprite`, `List`: name only, no box with internals
- the type still appears in `getSprite() : Sprite`

4: The Diagram (PlantUML)

▒▒▒ Just the classes and the relationships

```
@startuml CD-EXAMPLE
hide empty members

enum Direction <<enumeration>>
abstract class Unit
abstract class Square
class Pellet

' generalization: Pellet extends Unit
Unit <|-- Pellet

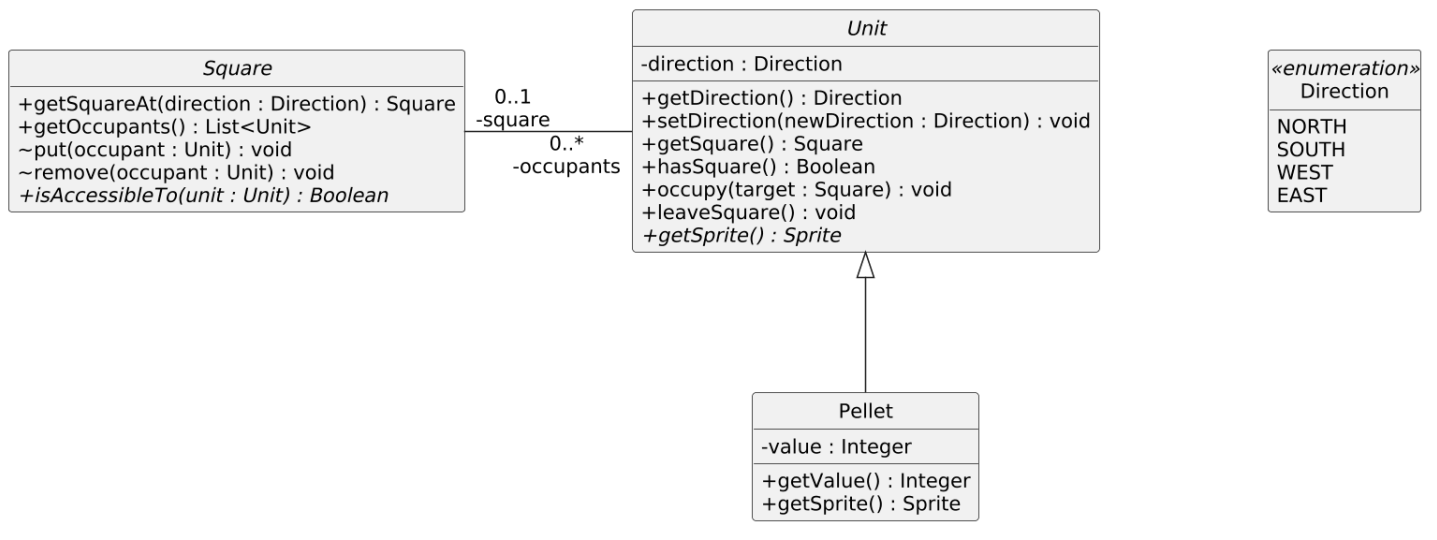
' bidirectional association, read from BOTH fields
Square "0..1\n-square" -right- "0..*\n-occupants" Unit
@enduml
```

Reading the last line, label by label:

- `"0..1\n-square"` sits at the Square end: it stands for the field `Unit.square`. `0..1` = a unit has none or one square · `-square` = the field's name (role), `-` = private
- `"0..*\n-occupants"` sits at the Unit end: it stands for the field `Square.occupants`. `0..*` = a square has zero or more units · `-occupants` = the field's name (role)
- `--` with no arrowheads = bidirectional (each class has a field to the other); `-right-` is `--` plus a layout hint

The full version (attributes + operations in each box) is the render on the next slide.

4: The Diagram (Render)



4: Cross-Check With IntelliJ (Task D)

Right-click the package → Diagrams → Show Diagram (IntelliJ Ultimate; free student licence at jetbrains.com/student).

▒▒ "There are no lines between the classes"

- By default the diagram shows boxes + inheritance only. Click Show Dependencies on the diagram toolbar to get the lines for fields and usages.
- Fields are hidden by default: turn on the Fields (and Methods) toolbar icons, then Change Visibility Level → private, or `- square`, `- occupants` won't appear.
- Opened from one class? It shows that class only → right-click the canvas → Add Class to Diagram. Opening a package adds every class in it; do `board`, `level`, `npc`, … one at a time, or add the missing classes by hand.

▒▒ What IntelliJ shows ≠ what you draw

- IntelliJ lists `square : Square` as a field in the box and draws a plain dependency line: no role, no multiplicity, no aggregation / composition.
- You still draw it the UML way (association + role + multiplicity) and write the difference in your reconcile table: "tool shows `board` as a field; drawn as an association, `1`".

Docs: https://www.jetbrains.com/help/idea/class-diagram.html

5: What the Example Does NOT Decide For You

Each needs evidence from the code, written next to the relationship in your report.

- Aggregation / composition (hollow ◇ / filled ◆ on the whole) A field alone is not evidence. Ask: who creates the part? does the part die with the whole? can the part be shared? No clear answer → plain association
- Dependency (dashed arrow) A class that only uses another (parameter, local variable, static call) with no field
- Interfaces («interface», dashed realization) Grep for `implements`; check nested interfaces inside classes too
- What to leave out Factories, parsers, loaders, private nested classes may be omitted, unless they appear in one of your sequence diagrams